

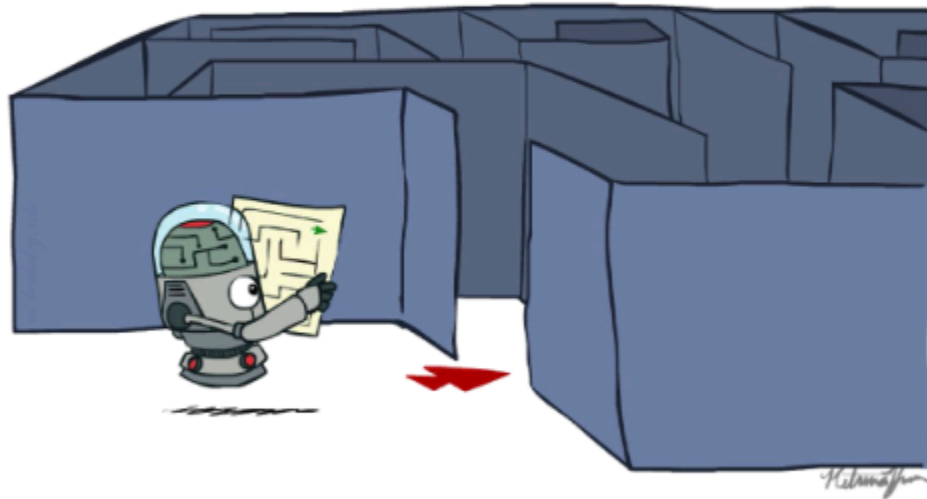
E quanto à pesquisa em IA?



<https://bair.berkeley.edu>

CS 188: Inteligência Artificial

Procurar



Instrutores: Peyrin Kao e Emma Pierson

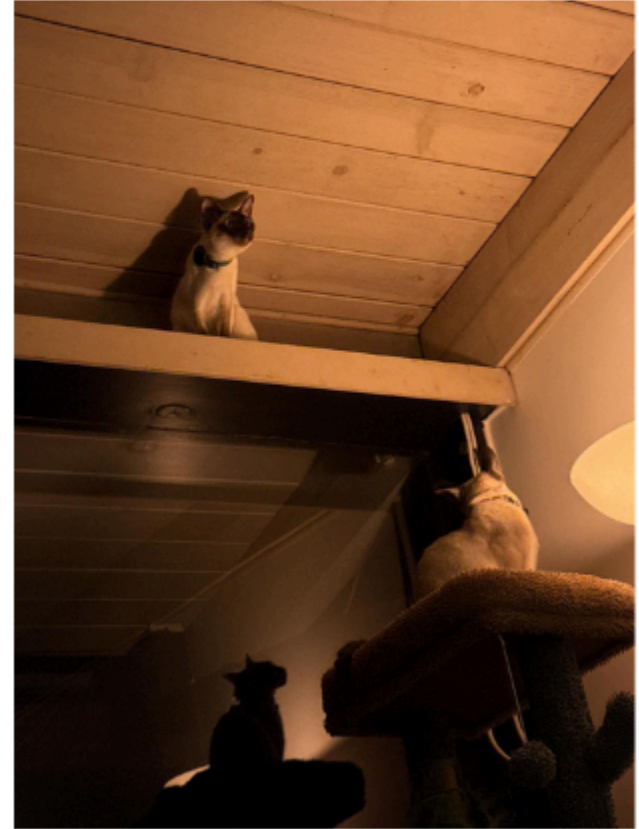
Universidade da Califórnia, Berkeley

[Esses slides foram criados por Dan Klein e Pieter Abbeel para o CS188 Introdução à IA na UC Berkeley (ai.berkeley.edu).]

Correção importante da aula anterior

Correção importante da aula anterior

- ChatGPT pode ser engraçado?



Correção importante da aula anterior

- ChatGPT talvez possa ser engraçado? ▪

Legenda sugerida: "Quando seu irmão ou irmã toma vantagem em uma discussão"

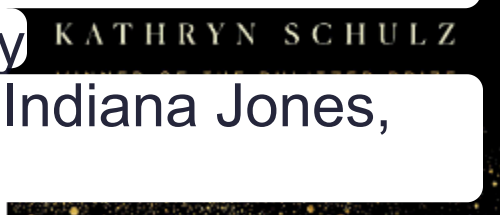


Por que nos importamos com buscas?

- Um problema onipresente no dia a dia: como você chegou a esse auditório? Como vai
Você chegou em casa? ▪ Quais disciplinas você deve fazer, em que ordem, para conseguir
Seus objetivos de vida?

Por que nos importamos com buscas?

"Histórias sobre procurar algo estão entre as histórias mais antigas, duradouras e populares que contamos. O objeto típico dessas narrativas de busca é algo de imenso valor escondido em algum lugar desconhecido ou distante. Assim como nas coisas reais que procuramos, esse objeto pode ser concreto ou abstrato, algo que o herói perdeu ou algo que ele ou ela nunca viu antes: Jason e os Argonautas procuraram por um veludo dourado, Psique procurou seu amante, Harry Potter procurou Horcruxes, e todos, de Galahad a Indiana Jones, procuraram o Santo Graal."



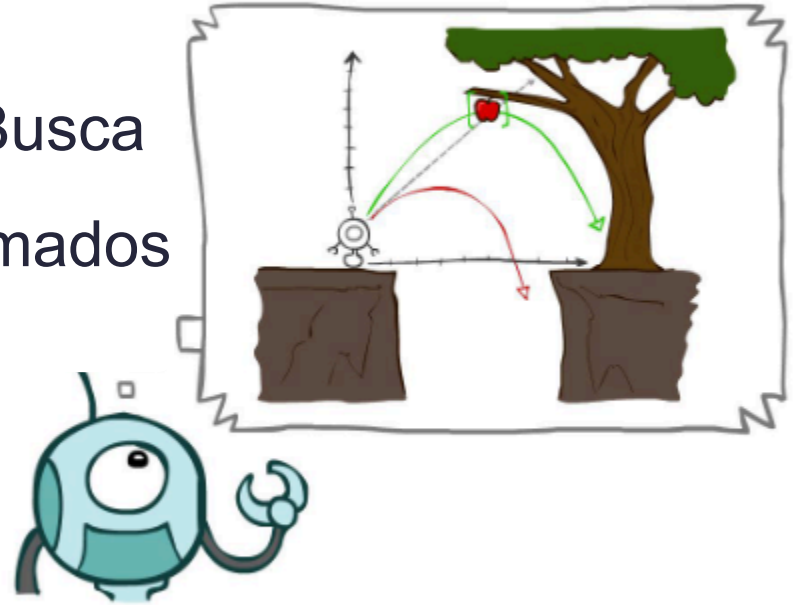
Por que nos importamos com buscas?

- Os algoritmos que vamos discutir são incrivelmente amplamente aplicados
 - Problemas de caminho/roteamento (tanto para pessoas quanto robôs)
 - Projeto de chips (fios de roteamento)
 - Problemas de planejamento de recursos
 - Planejamento de movimento robótico
 - Design de proteínas
 - Análise de linguagem
 - Tradução automática
 - Videogames
 - Reconhecimento de fala

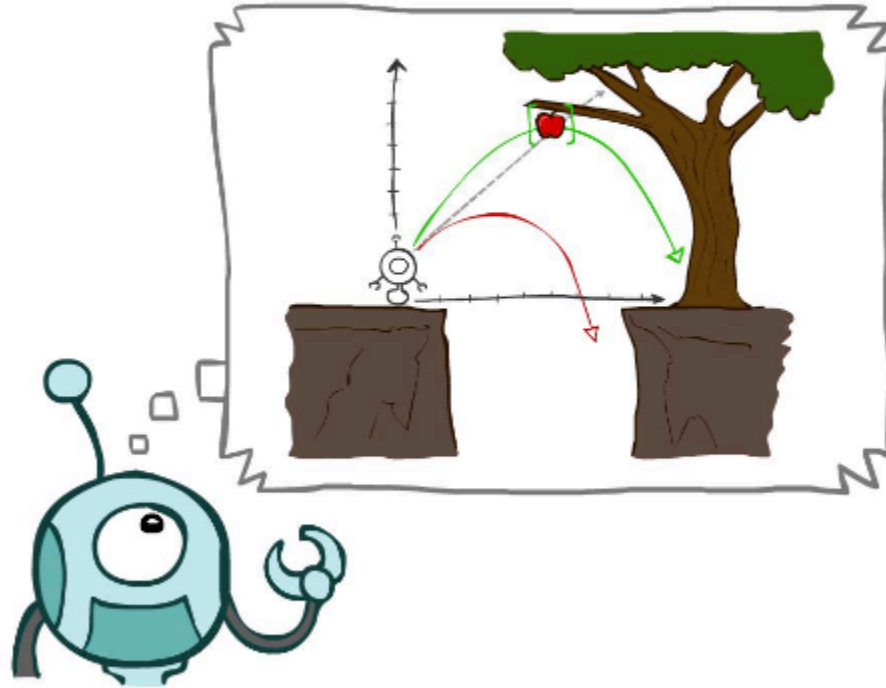
Hoje

- Agentes que Planejam Com Antecedência
- Problemas de Busca
- Métodos de Busca Pouco Informados

- Busca em profundidade
- em profundidade
- Busca em larga largura e custo uniforme

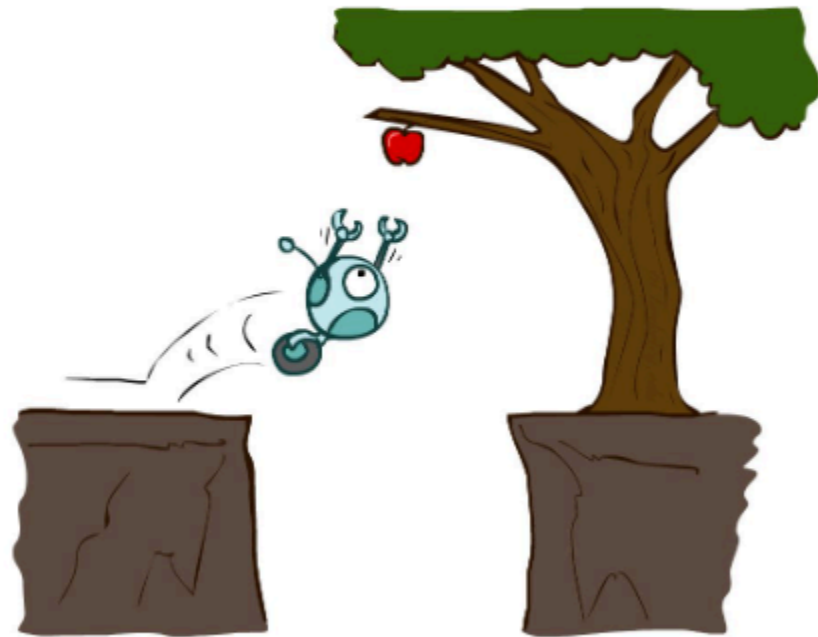


Agentes que Planejam



Agentes Reflexos

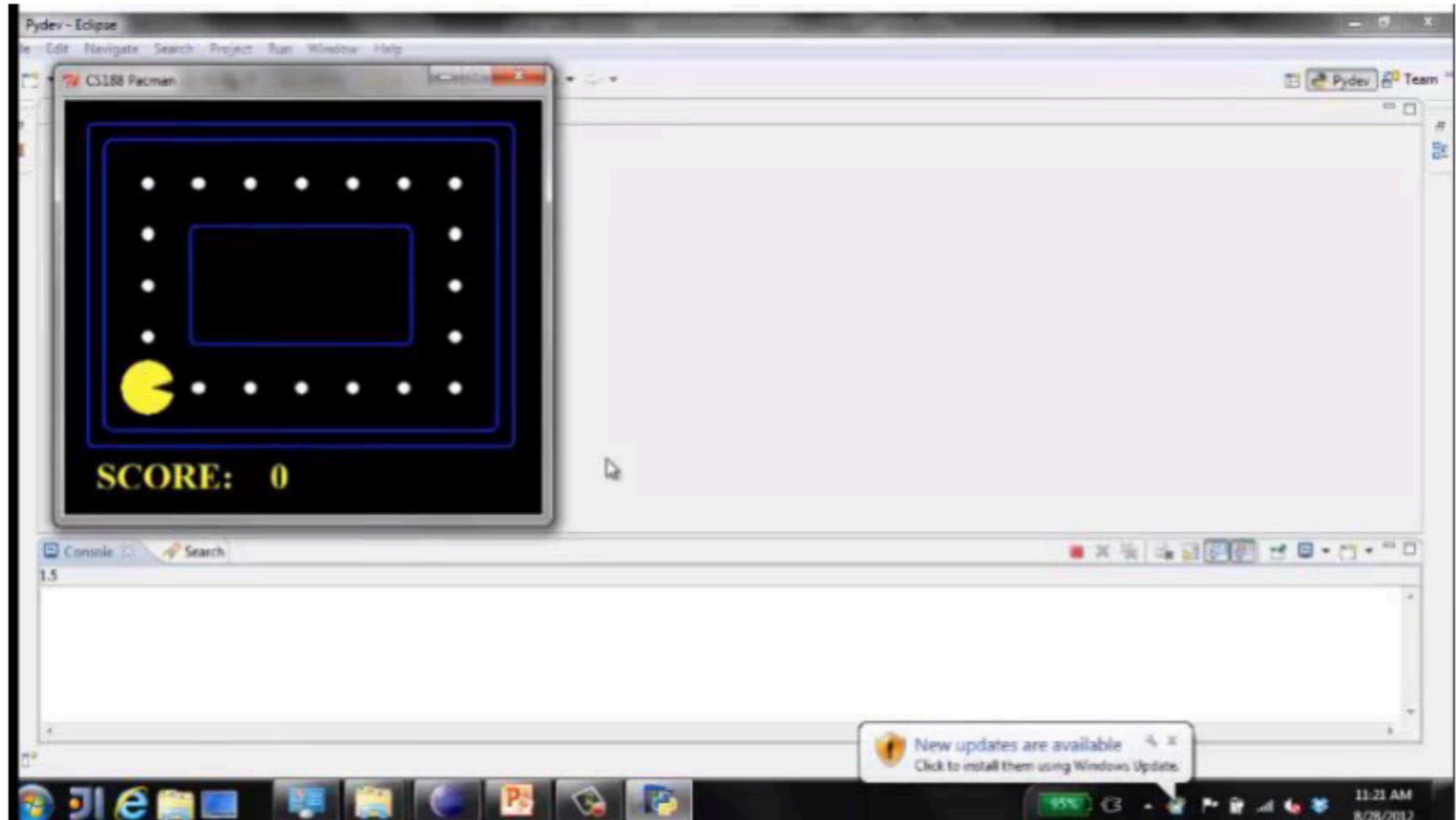
- Agentes reflexos: ▪ Escolham a ação com base na percepção atual (e talvez memória) ▪ Pode ter memória ou um modelo do mundo
- Estado▪ atual Não considere as consequências futuras de suas ações▪ Considere como o mundo É
- Um agente reflexo pode ser racional?



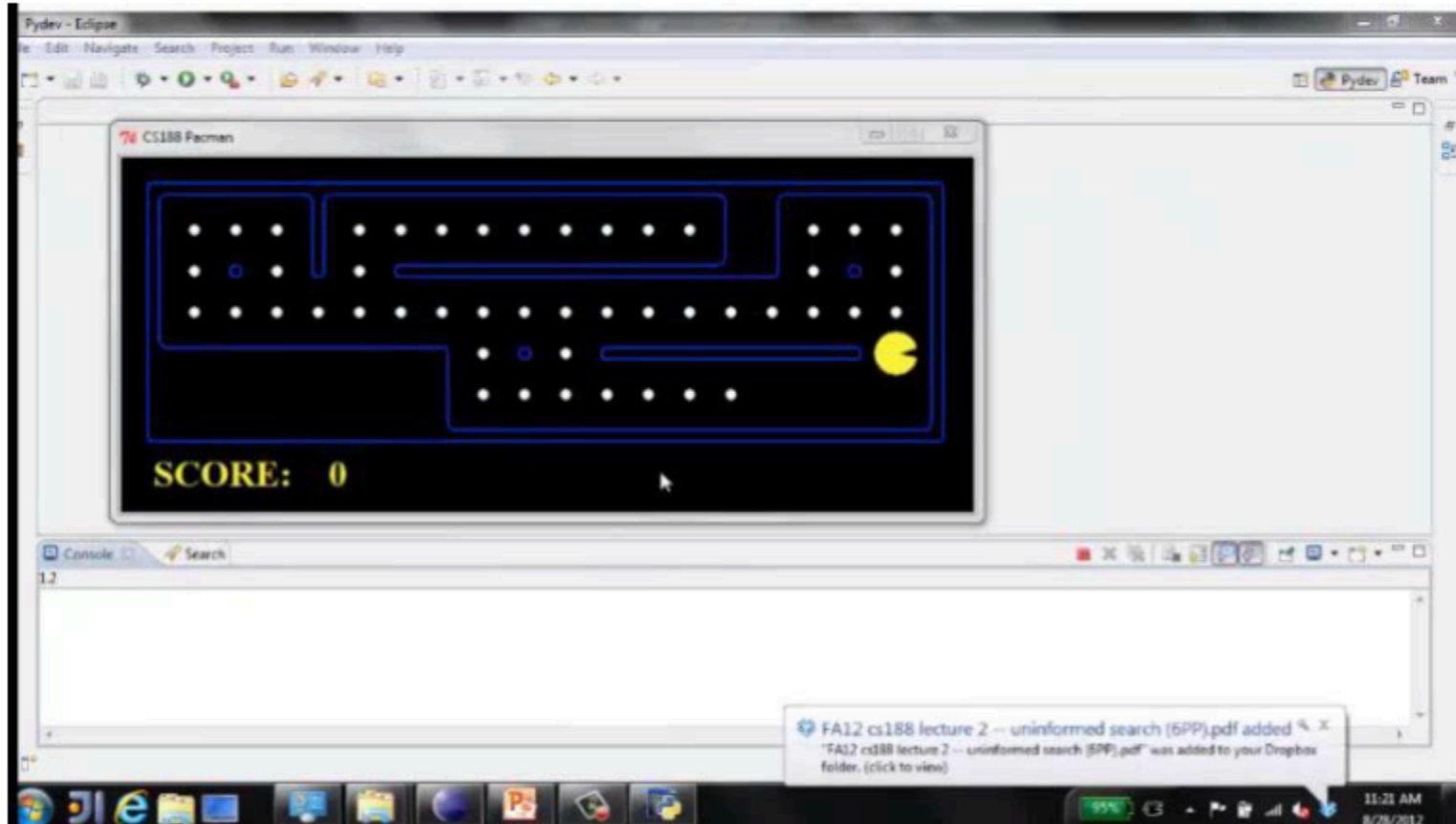
[Demo: reflexo ótimo (L2D1)]

[Demo: reflexo ótimo (L2D2)]

Vídeo da Demo Reflex Optimal

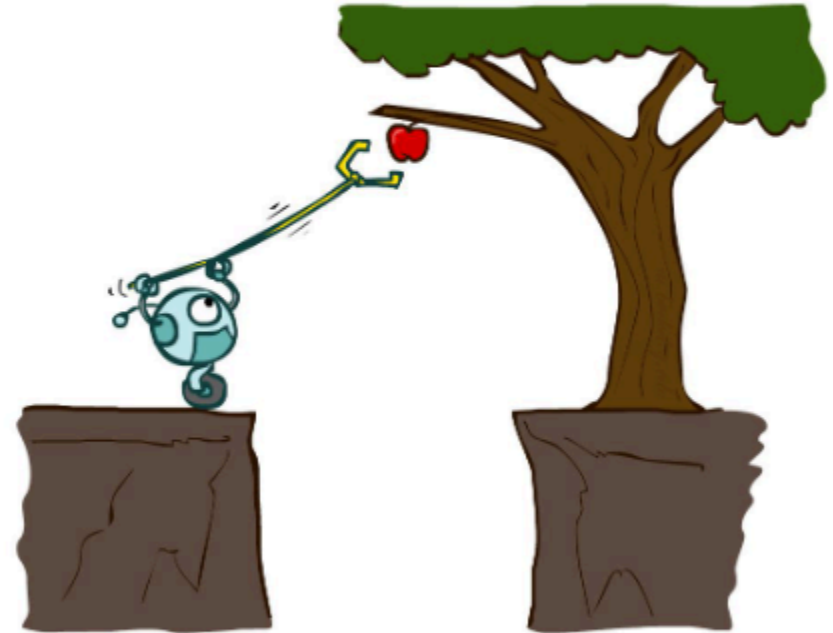


Vídeo do Demo Reflex Odd



Agentes de Planejamento

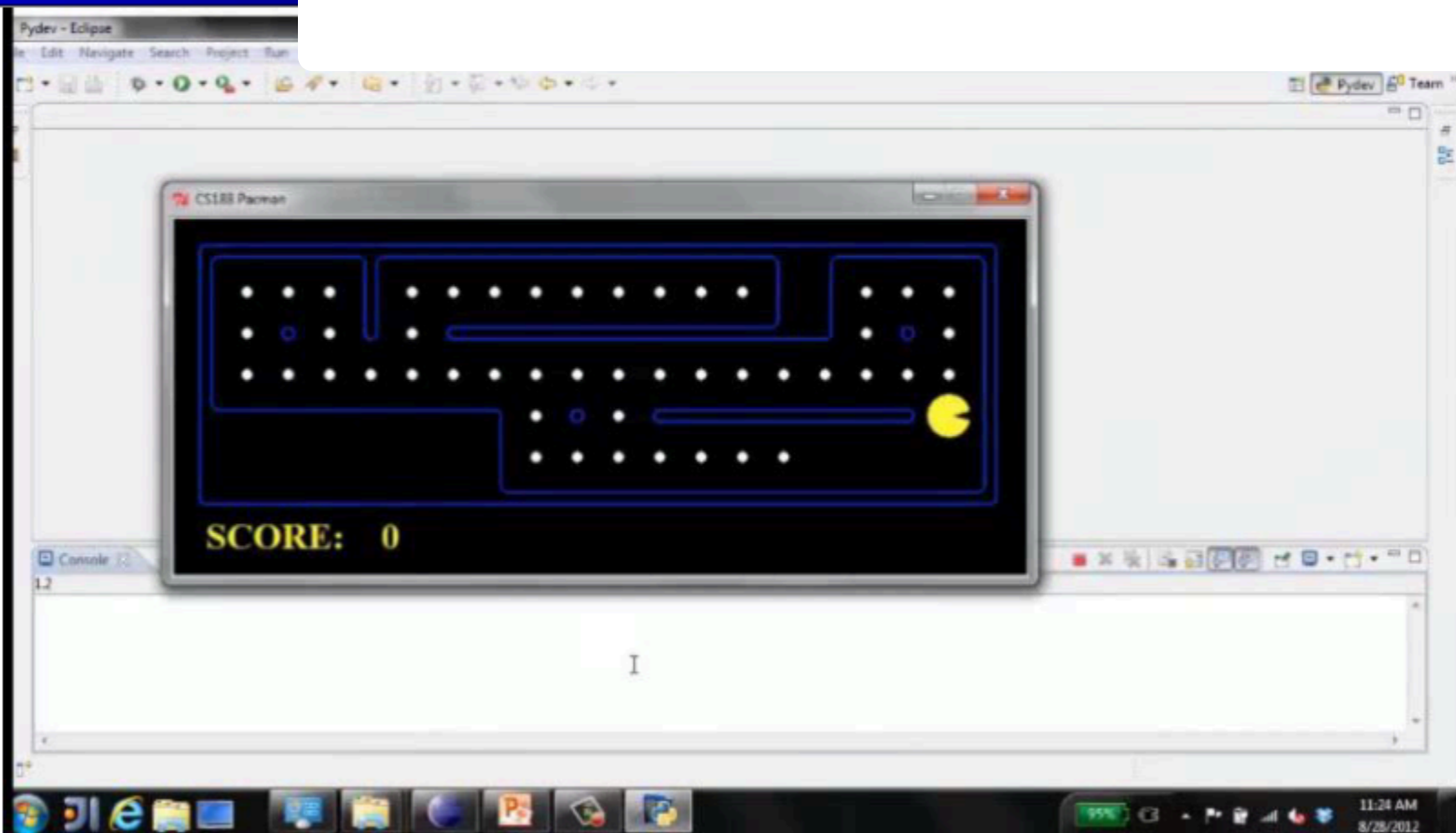
- Agentes de planejamento: ▪ Perguntem decisões do tipo "e se" ▪ baseadas em (hipotetizadas) consequências das ações ▪ Deve ter um modelo de como o mundo evolui em resposta às ações ▪ Deve formular um objetivo (teste) ▪ Considerar como o mundo SERIA
- Planejamento ótimo vs. planejamento completo Planejamento vs. replanejamento



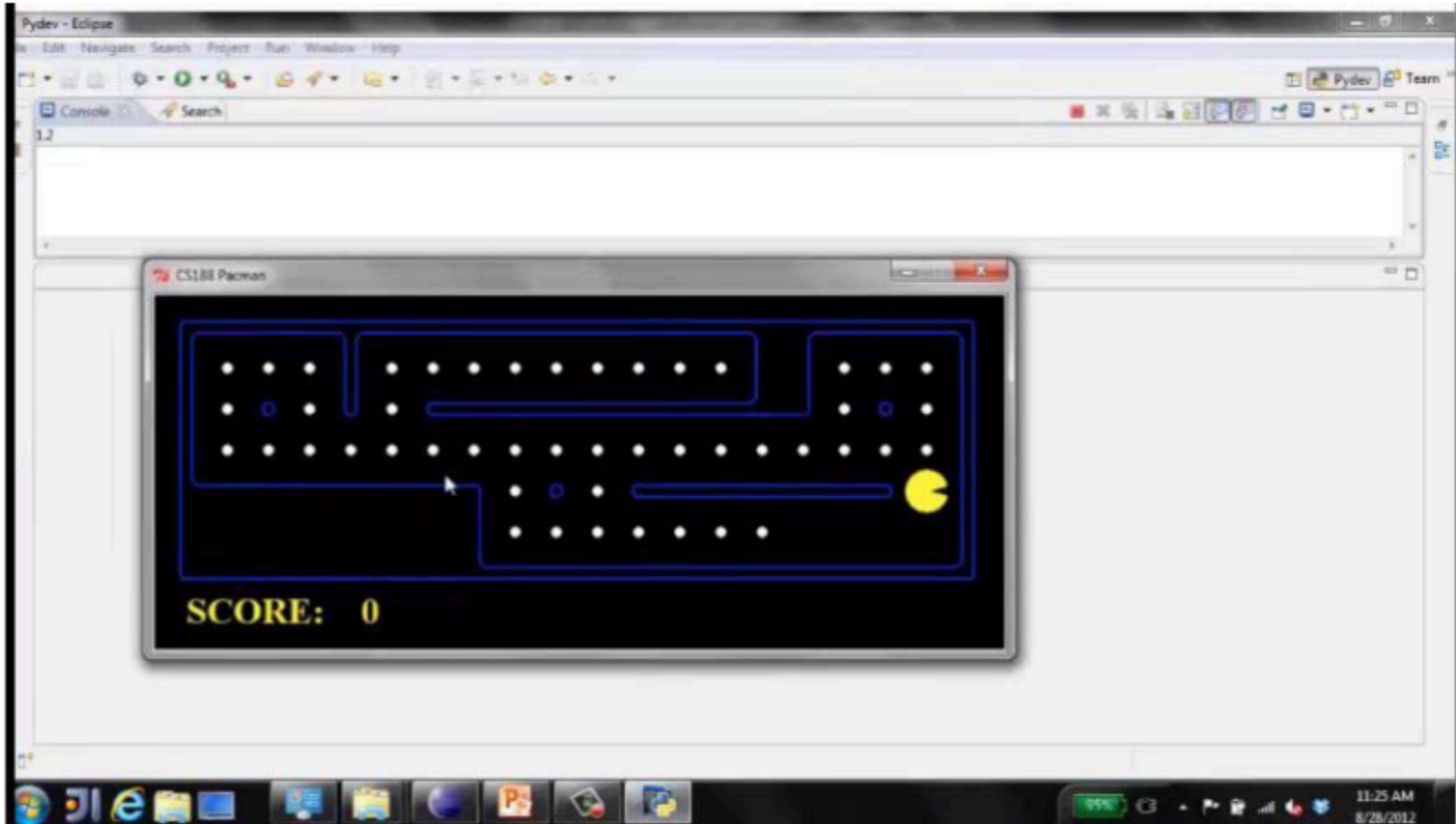
[Demo: replanejamento (L2D3)]

[Demo: mastermind (L2D4)]

Vídeo de Replanejamento da Demo



Vídeo do Demo Mastermind



Problemas de busca



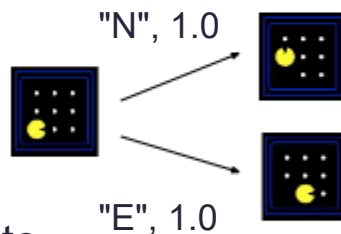
Problemas de busca

- Um problema de busca consiste em:

- Um espaço de estados

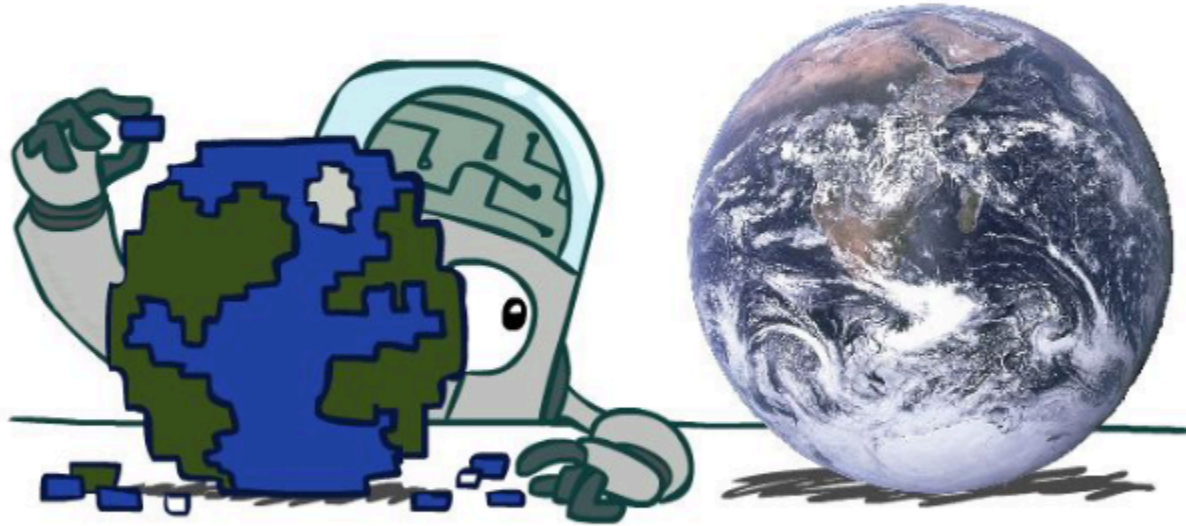


- Uma função sucessora
(com ações, custos)

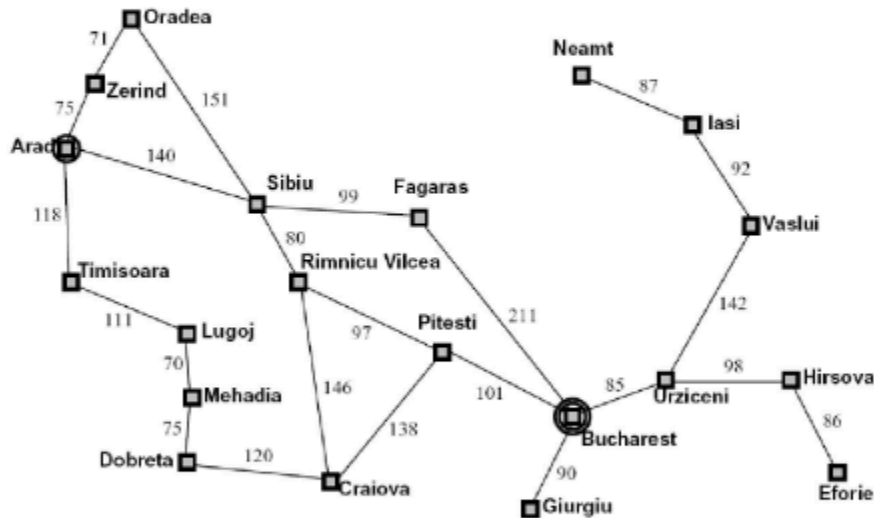


- Um estado de início e um teste de meta
- Uma solução é uma sequência de ações (um plano) que transforma o estado inicial em um estado objetivo

Problemas de busca são modelos



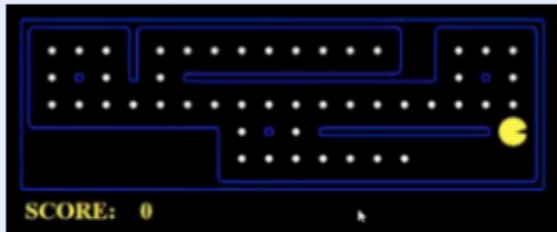
Exemplo: Viajando na Romênia



- Espaço de estados:
 - Cidades
- Função sucessora:
 - Estradas: Vá para a cidade vizinha com custo = distância
- Estado inicial:
 - Arad
- Teste de objetivo:
 - É o estado == Bucareste?
- Solução?

O que há em um espaço de estados?

O estado mundial inclui cada detalhe do ambiente



Um estado de busca mantém apenas os detalhes necessários para o planejamento (abstração)

▪ Problema: Caminho

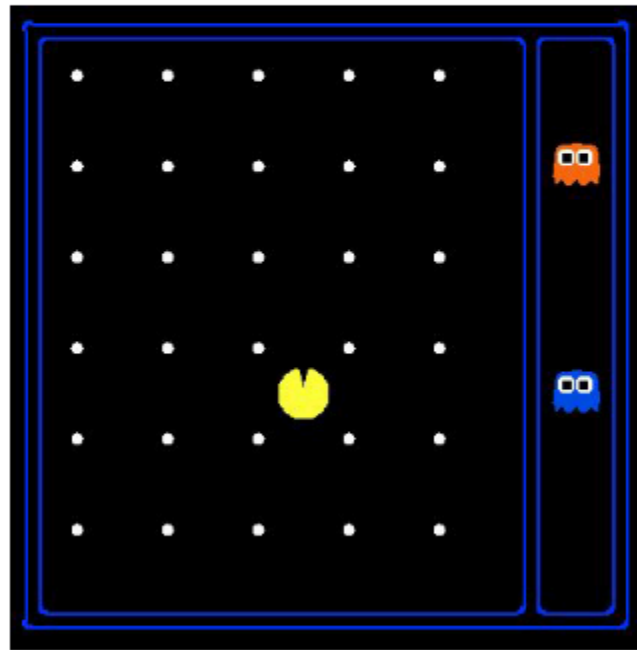
- Estados: (x,y) localização
- Ações: NSEW
- Sucessor: atualizar localização apenas
- teste de objetivo: é $(x,y)=FIM$

▪ Problema: Comer Tudo-Os Pontos

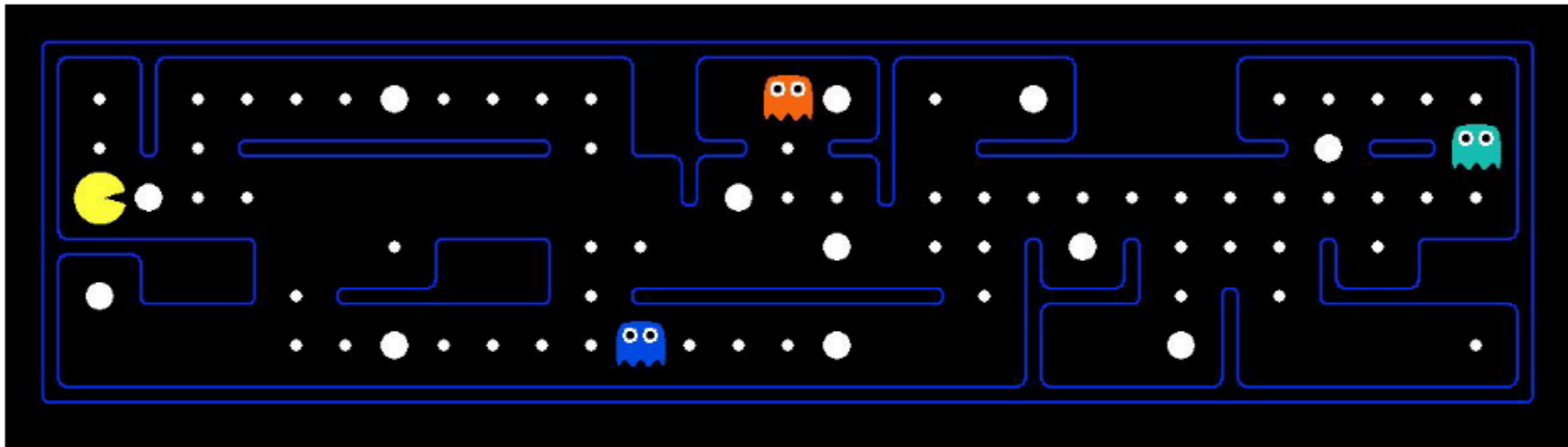
- Estados: $\{(x,y), \text{booleanos de ponto}\}$
- Ações: NSEW
- Sucessor: local de atualização e possivelmente um teste de objetivo booleano
- de pontos: pontos todos falsos

Tamanhos dos espaços de estados?

- Estado mundial:
 - Posições de agentes: 120
 - Contagem de comida: 30
 - Posições fantasmas: 12
 - Frente para agente: NSEW
- Quantos
 - Estados do mundo?
 - $120 \times (230) \times (122) \times 4$
 - estados para caminhos?
 - 120 estados para comer tudo-tudo?
 - $120 \times (230)$

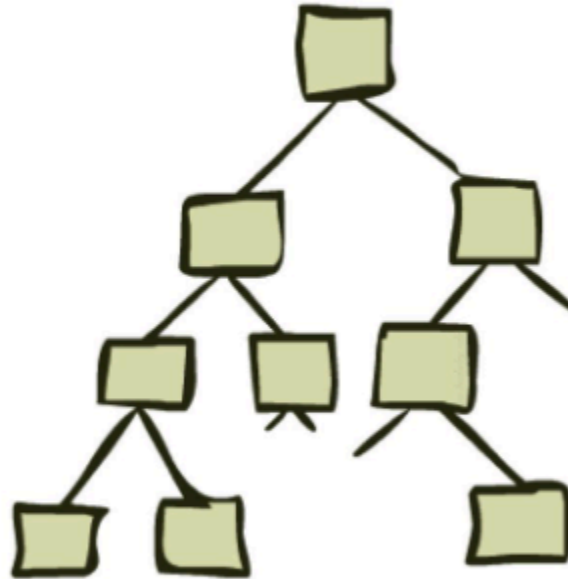


Quiz: Passagem Segura



- Problema: devora todos os pontos enquanto mantém os fantasmas permanentemente assustados. O que o espaço de estados precisa especificar?
 - (posição do agente, booleanos do ponto de ponto, booleanos do pellet de energia, tempo de medo restante)

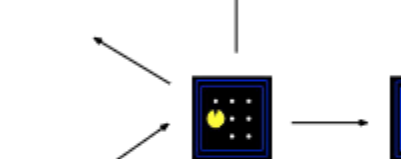
Grafos de espaço de estados e árvores de busca

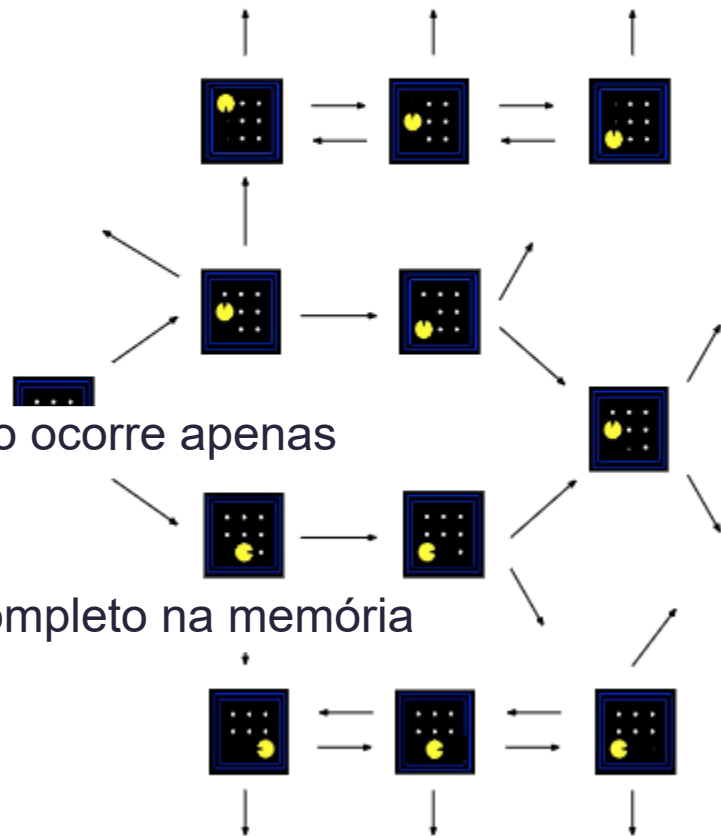


Grafos em espaço de estados

- Grafo do espaço de estados: Um matemático

Representação de um problema de busca

- Nós são configurações de mundo (abstratas). ▪ Arcos representam sucessores (resultados de ação). ▪ O teste objetivo é um conjunto de nós objetivo (talvez apenas um)
 - Em um grafo de espaço de estados, cada estado ocorre apenas uma vez!
 - Raramente conseguimos construir esse grafo completo na memória (é grande demais), mas é uma ideia útil
- 
- The diagram illustrates a state space graph. It features several nodes, each represented as a 3x3 grid. Some grids contain a yellow dot in a specific position, representing a state. Arrows connect these grids, indicating transitions between states. The graph is partially visible, showing a branching structure from a central node.

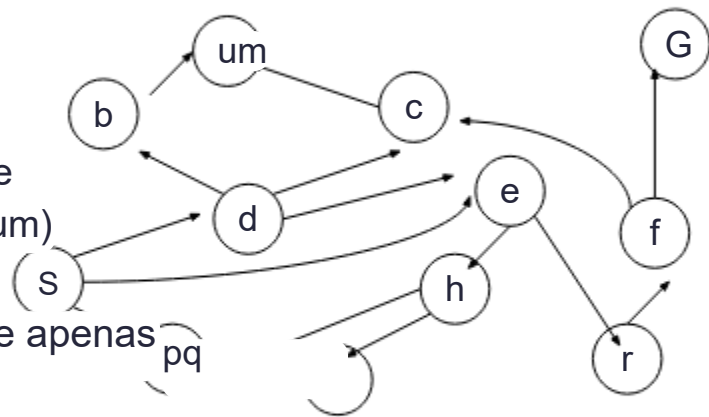


Grafos em espaço de estados

- Grafo do espaço de estados: Um matemático

Representação de um problema de busca

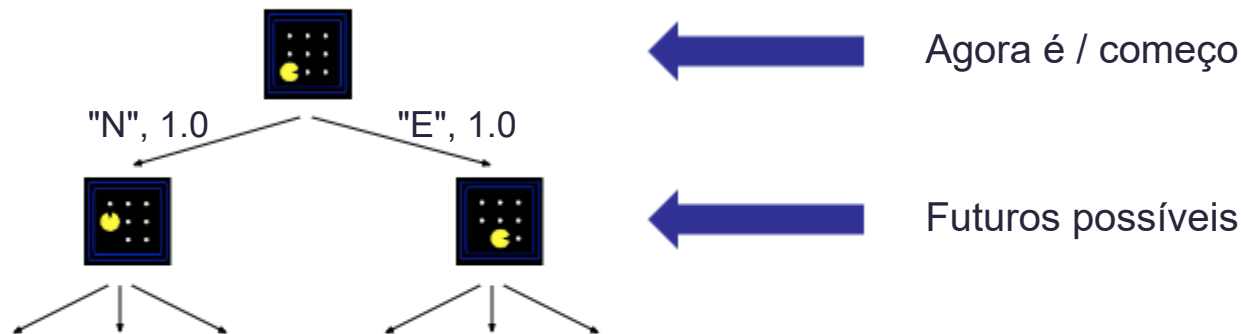
- Nós são configurações de mundo (abstratas).
 - Arcos representam sucessores (resultados de ação).
 - O teste objetivo é um conjunto de nós objetivo (talvez apenas um)
- Em um grafo de espaço de estados, cada estado ocorre apenas uma vez!
- Raramente conseguimos construir esse grafo completo na memória (é grande demais), mas é uma ideia útil



Grafo de espaço de estados minúsculo para um espaço de estados minúsculo

Problema de busca

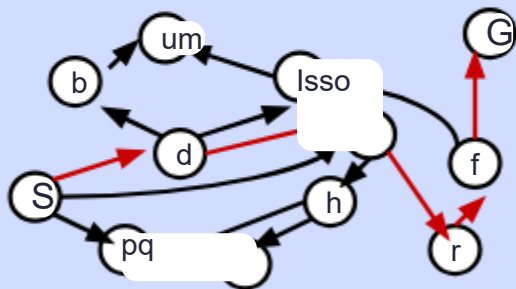
Árvores de Busca



- Uma árvore de busca:
 - Uma árvore "e se" de planos e seus resultados
 - O estado inicial é o nó raiz
 - Filhos correspondem a sucessores
 - Nós mostram estados, mas correspondem a PLANOS que alcançam esses estados
 - Para a maioria dos problemas, nunca conseguimos construir a árvore inteira

Grafos de espaço de estados vs. árvores de busca

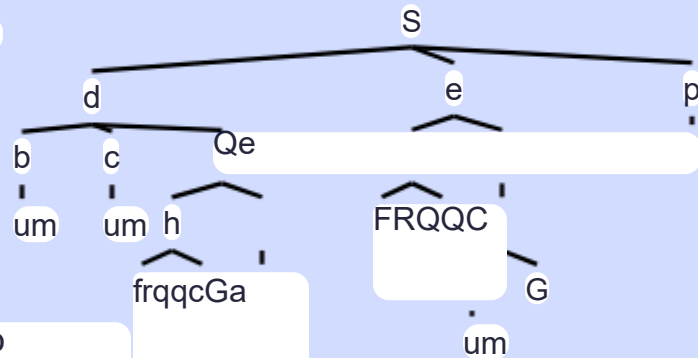
Grafo de Espaço de Estado



Cada NÓ em na
árvore de busca é um
CAMINHO inteiro em
O espaço de estados
gráfico.

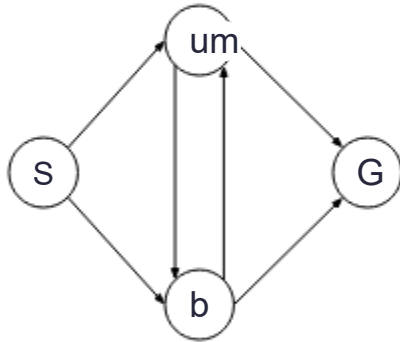
Construímos ambos
sob demanda – e
Construímos o mínimo
possível.

Árvore de Busca



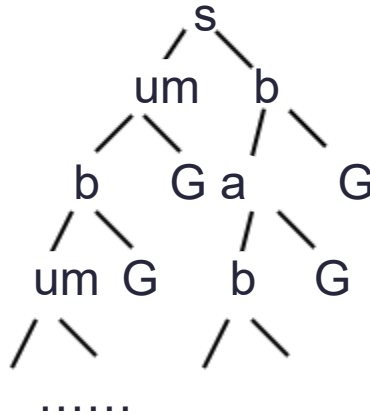
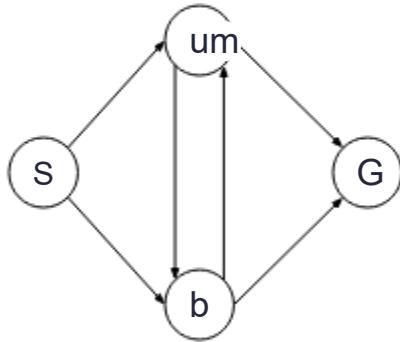
Quiz: Gráficos de Espaço de Estado vs. Árvores de Busca

Considere este gráfico de 4 estados: Qual o tamanho da árvore de busca dela (a partir de S)?



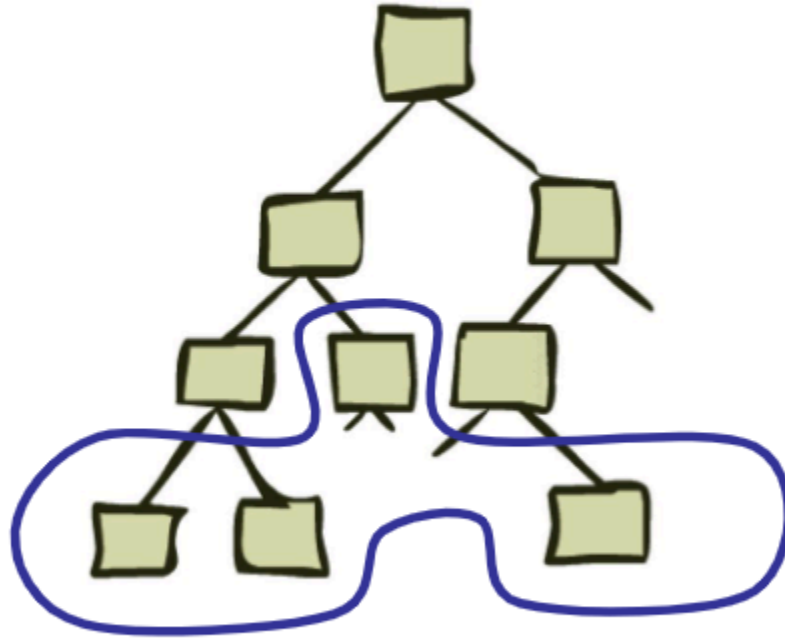
Quiz: Gráficos de Espaço de Estado vs. Árvores de Busca

Considere este gráfico de 4 estados: Qual o tamanho da árvore de busca dela (a partir de S)?

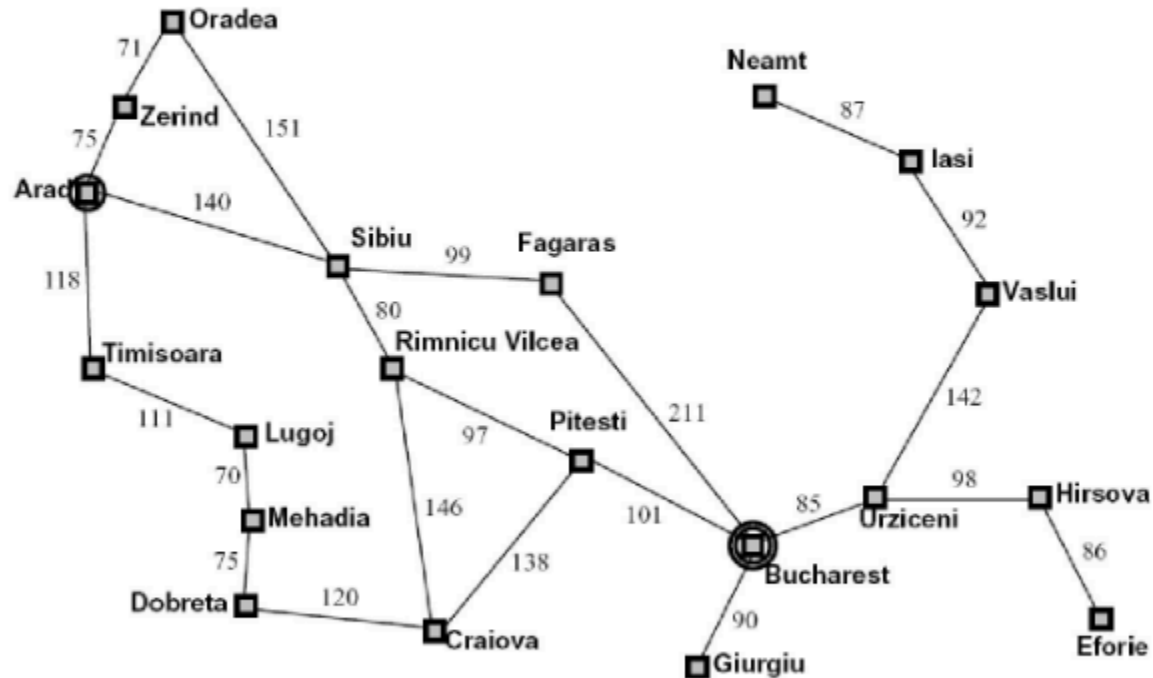


Importante: Muita estrutura repetida na árvore de busca!

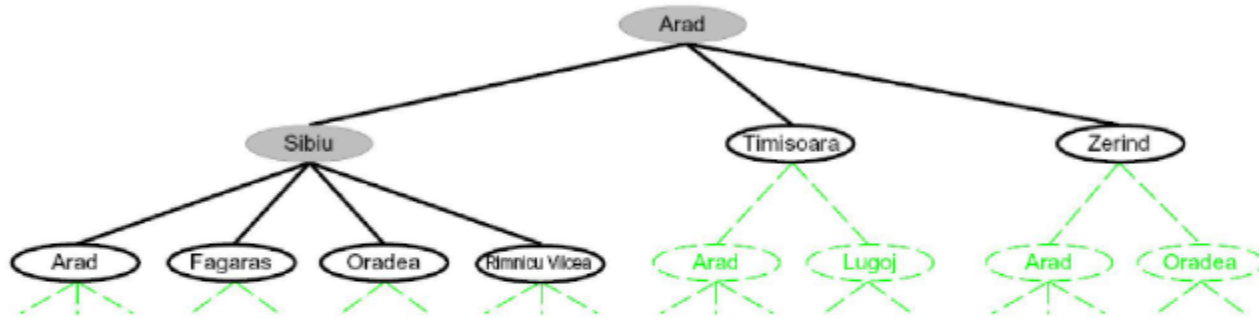
Busca em Árvores



Exemplo de Pesquisa: Romênia



Pesquisando com uma Árvore de Busca



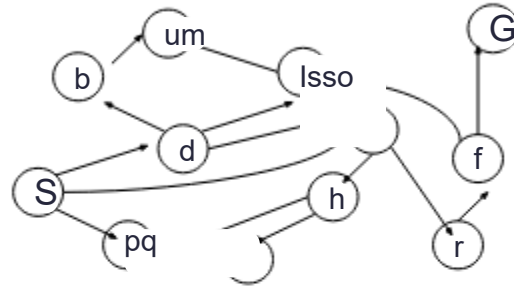
- Busca:
- Expandir planos potenciais (nós árvore)
- Manter uma margem de planos parciais em consideração
- Tentar expandir o menor número possível de nós árvore

Busca Geral por Árvores

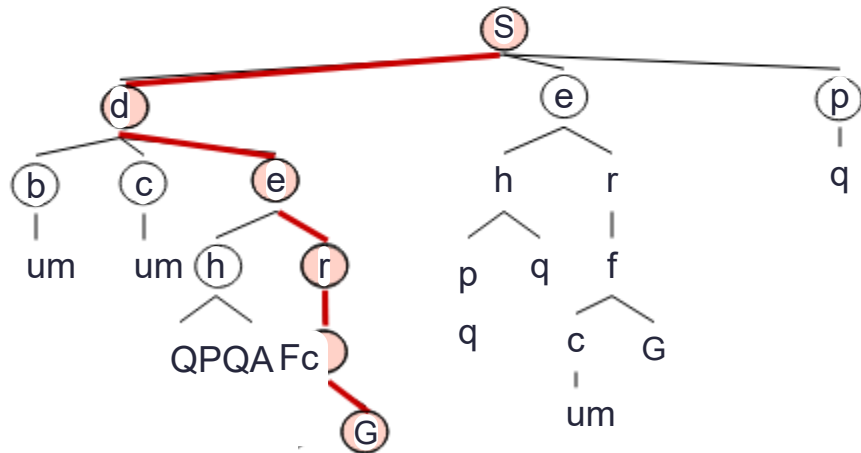
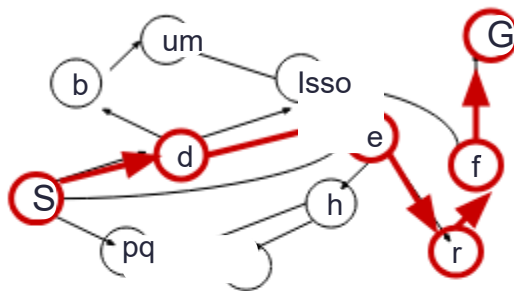
```
function TREE-SEARCH( problem, strategy ) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```

- Ideias importantes: ▪ Estratégia ▪ de Exploração da Expansão ▪ da Franja ▪ Pergunta principal: quais nós de franja explorar?

Exemplo: Busca por Árvore



Exemplo: Busca por Árvore



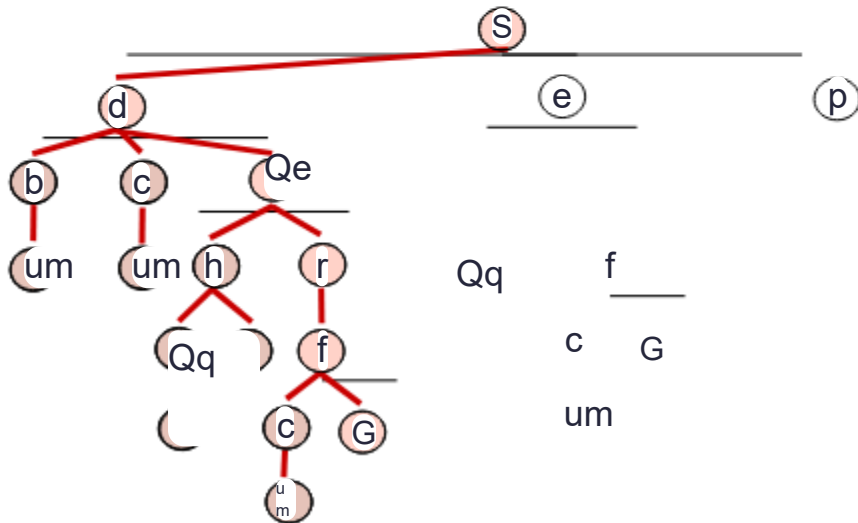
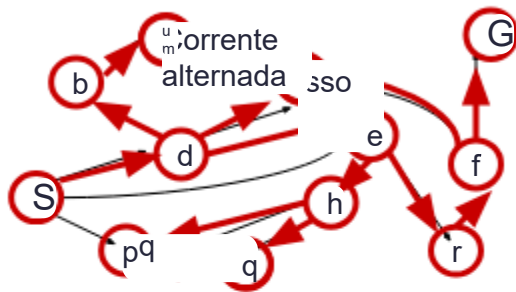
-ss → ds → es → ps
 -→ d → bs → d →
 cs → d → es → d
 → e → hs → d → e
 → rs → d → e → r
 → fs → e → r → f
 → cs → d → e → r
 → f → G

Busca em profundidade

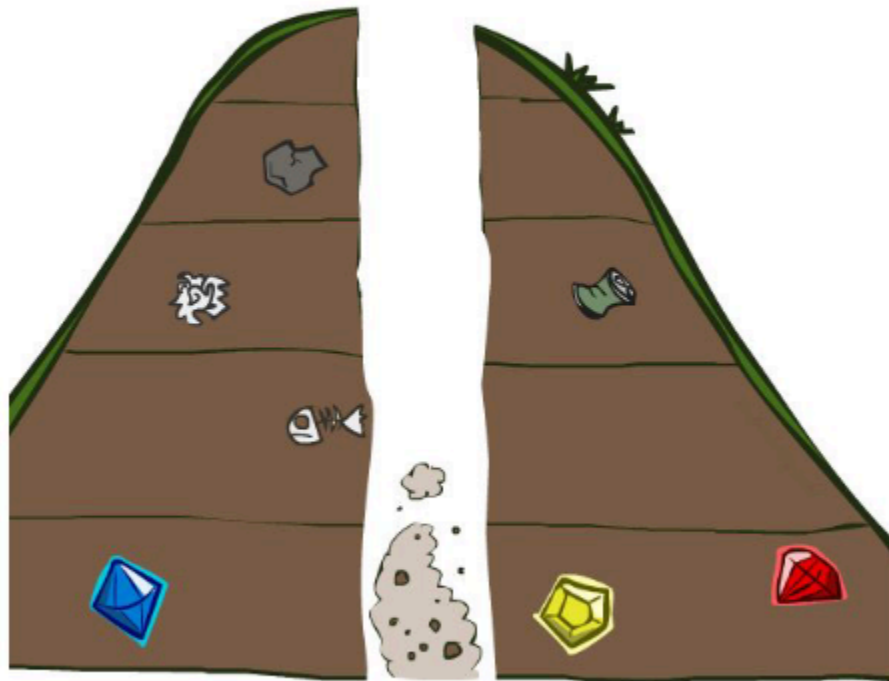


Busca em profundidade

Estratégia: expandir
primeiro o nó mais
profundo Implementação:
Fringe é uma pilha LIFO



Propriedades do Algoritmo de Busca



Propriedades do Algoritmo de Busca

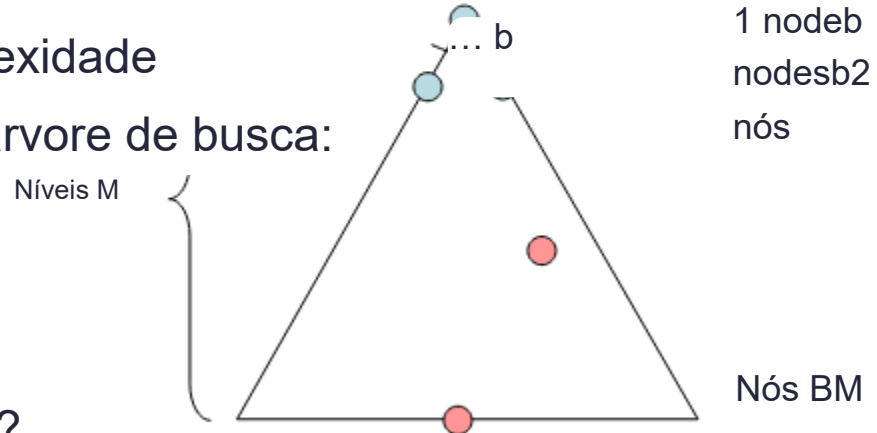
- Completo: Garantido para encontrar uma solução, se existir?
- Ótimo: Garantido para encontrar o caminho de menor custo?

Complexidade de tempo? ▪ Complexidade espacial? ▪ Desenho animado da árvore de busca:

- b é o fator de ramificação
- m é a solução de profundidade máxima em várias profundidades

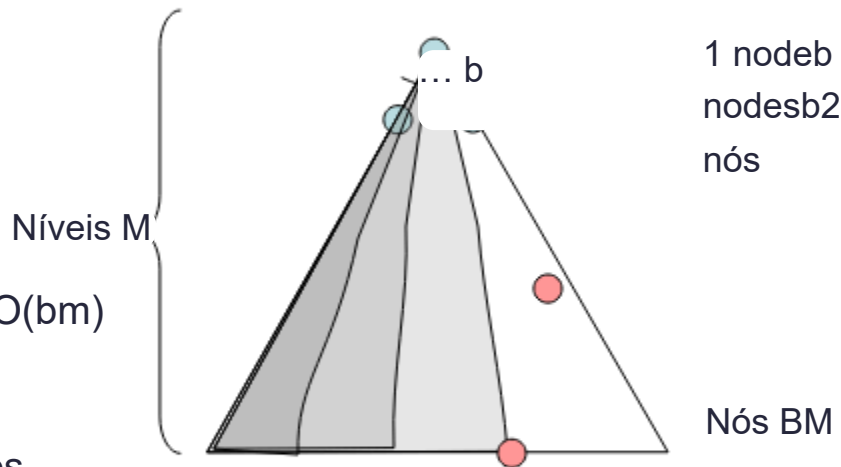
▪ Número de nós em toda a árvore?

- $1 + b + b^2 + \dots + b^m = O(b^{m+1})$



Propriedades de Busca em Profundidade (DFS)

- Quais nós o DFS expande?
 - Algum prefixo à esquerda da árvore.
 - Poderia processar toda a árvore! ▪
Se m é finito, leva tempo $O(bm)$
- Quanto espaço a franja ocupa?
 - Só tem irmãos no caminho até a raiz, então $O(bm)$
- Está completo?
 - m poderia ser infinito, então só se impedirmos Ciclos (mais informações depois)
- É o ideal?
 - Não, ele encontra a solução "mais à esquerda", Independentemente da profundidade ou custo

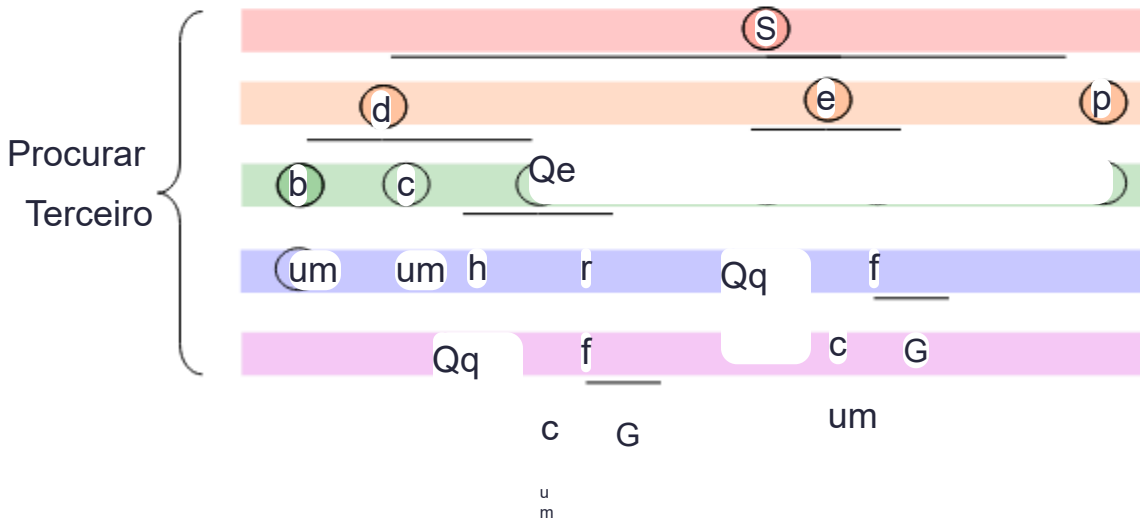
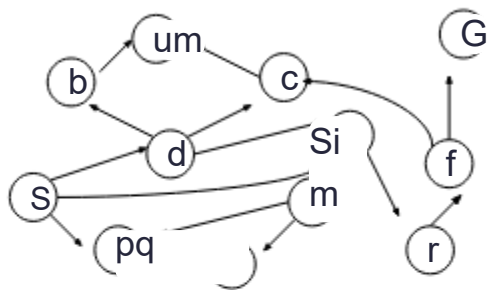


Busca em Largura



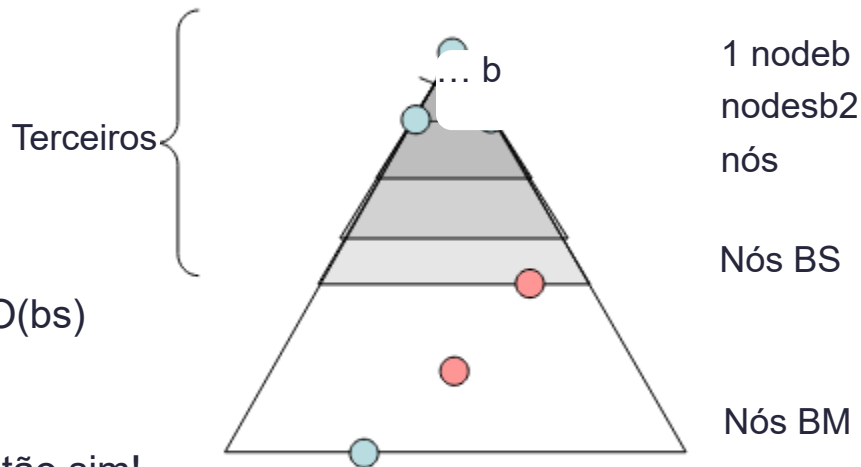
Busca em Largura

Estratégia: expandir
primeiro o nó mais
básico Implementação:
Fringe é uma fila FIFO



Propriedades de Busca em Largura (BFS)

- Quais nós o BFS expande?
 - Processa todos os nós acima da solução
 - mais rasa Seja a profundidade da solução mais rasa s
 - A busca leva tempo $O(b^s)$
- Quanto espaço a franja ocupa?
 - Tem mais ou menos o último nível, então $O(b^s)$
- Está completo?
 - s deve ser finito se existe uma solução, então sim!
- É o ideal?
 - Somente se os custos forem todos 1 (falarei sobre custos depois)



Quiz: DFS vs BFS



Quiz: DFS vs BFS

- Quando o BFS vai superar o DFS?
- Quando o DFS vai superar o BFS?

Vídeo do DFS/BFS de Demonstração Maze Water (parte 1)



Vídeo da demonstração do Maze Water

DFS/BFS (parte 2)

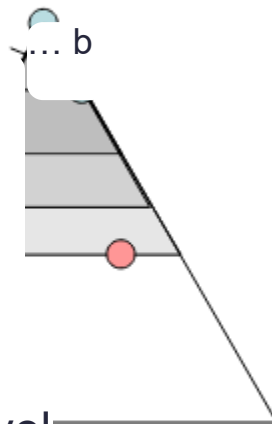


Aprofundamento Iterativo

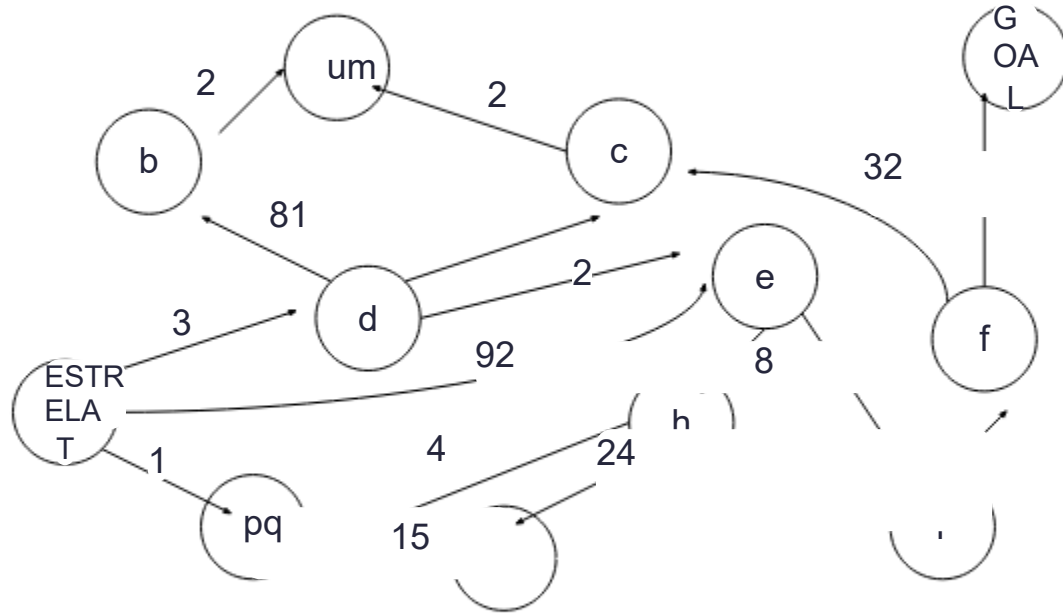
- Ideia: aproveite a vantagem de espaço do DFS com o BFS

Vantagens de tempo / solução superficial

- Execute um DFS com limite de profundidade 1. Se não houver solução... ▪ Execute um DFS com limite de profundidade 2. Se não houver solução... ▪ Execute um DFS com limite de profundidade 3.
 - Isso não é um desperdício redundante?
 - Geralmente, a maior parte do trabalho acontece no menor nível
- Pesquisado por nível, então não é tão ruim assim!



Busca sensível a custos



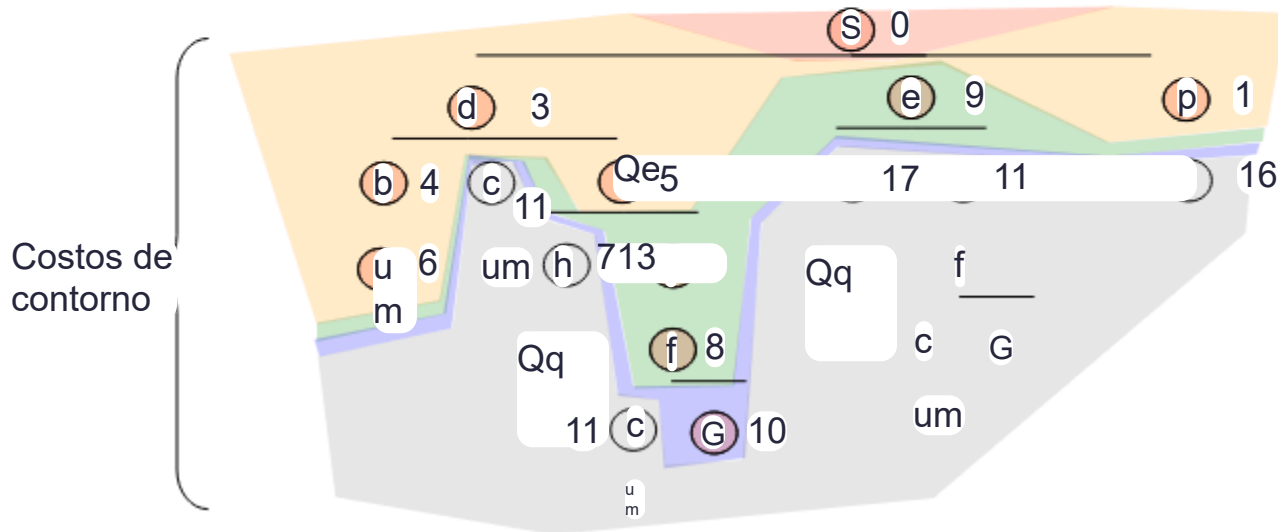
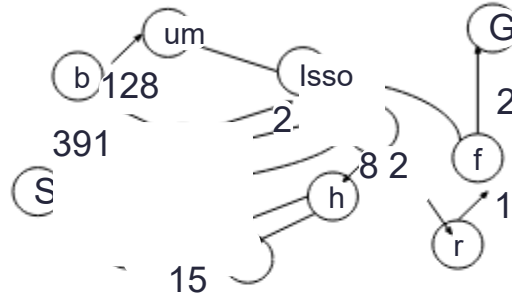
O BFS encontra o caminho mais curto em termos de número de ações.
Não encontra o caminho de menor custo. Agora vamos abordar um algoritmo semelhante que encontra o caminho de menor custo.

Busca de Custos Uniformes



Busca de Custos Uniformes

Estratégia: expandir primeiro o nó mais barato: A franja é uma fila de prioridade (prioridade: custo acumulado; números em árvore referem-se ao custo acumulado)



Propriedades da Busca Uniforme de Custos (UCS)

- Quais nós a UCS expande?

- Processa todos os nós com custo menor que a solução mais barata!
- Se essa solução custa C^* e arcos custam pelo menos ϵ , então o "profundidade efetiva" é aproximadamente C^*/ϵ . Leva tempo $O(bC^*/\epsilon)$ (exponencial em profundidade efetiva)

- Quanto espaço a franja ocupa?

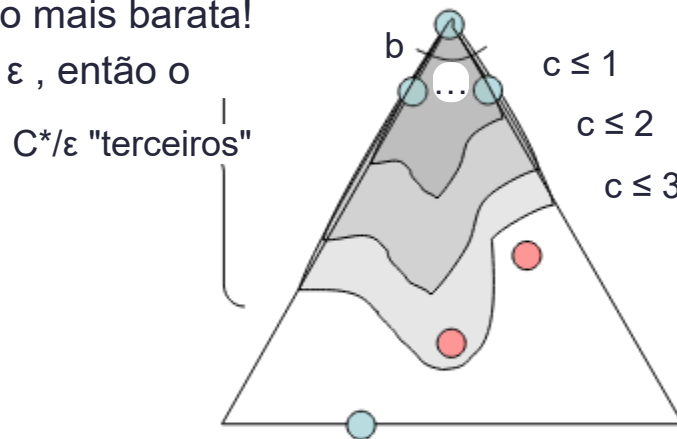
- Tem aproximadamente o último nível, então $O(bC^*/\epsilon)$

- Está completo?

- Assumindo que a melhor solução tem custo finito e custo mínimo de arco é positivo, sim!

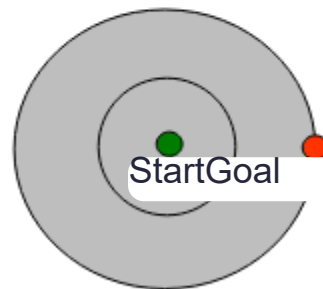
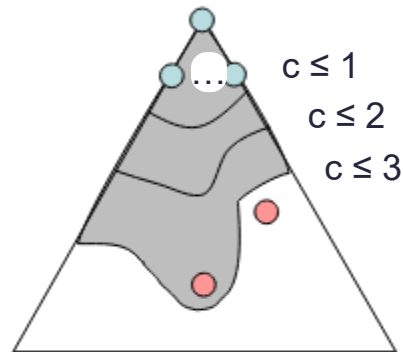
- É o ideal?

- Sim! (Prova na próxima aula via A^*)



Questões de Custo Uniforme

- Lembre-se: a UCS explora o aumento dos custos de contorno
- O lado bom: a UCS está completa e ideal!
- O ruim: ▪ Explora opções em todas as "direções" ▪ Sem informações sobre localização do gol
- Vamos resolver isso em breve!



[Demo: grade vazia UCS (L2D5)]

[Demo: labirinto com DFS/BFS/UCS
de águas profundas/rasas (L2D7)]

Vídeo da Demo Empty UCS



Vídeo de Demo Maze com Águas Profundas/Rasas --- DFS, BFS ou UCS? (parte 1)



Vídeo de Demo Maze com Águas Profundas/Rasas --- DFS, BFS ou UCS? (parte 2)



Vídeo de Demo Maze com Águas Profundas/Rasas --- DFS, BFS ou UCS? (parte 3)



A Fila Única

- Todos esses algoritmos de busca são os

O mesmo, exceto pelas estratégias marginais

- Conceitualmente, todas as franjas são prioridade

Filas (ou seja, coleções de nós com prioridades anexadas) ▪ Na prática,

para DFS e BFS, você pode

evitar o overhead $\log(n)$ de uma fila de prioridade efetiva, usando filas ▪

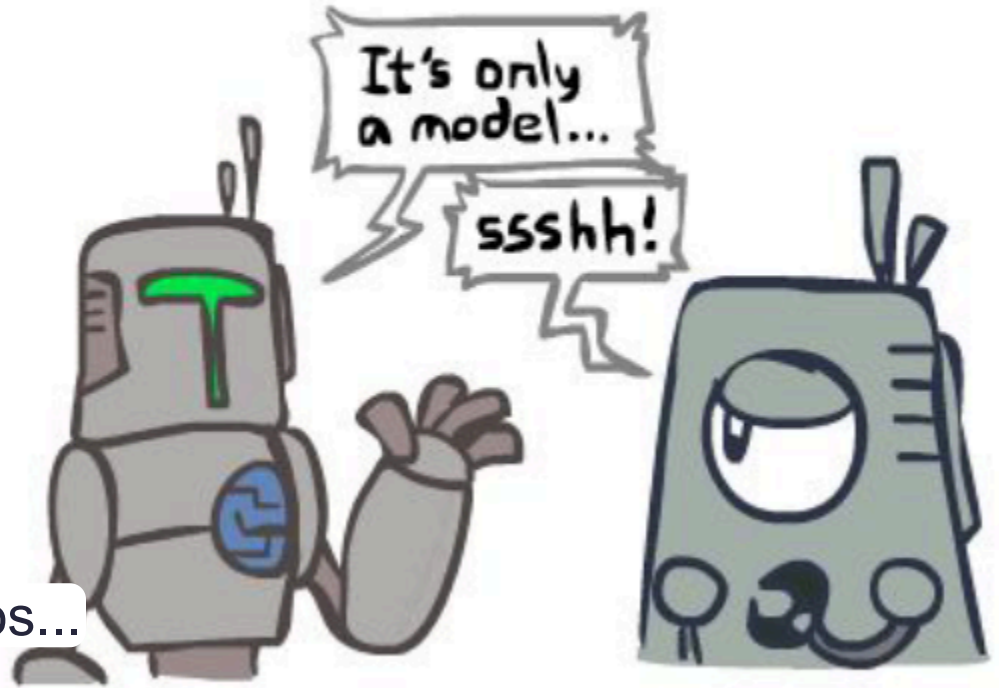
stacks and Pode até programar uma implementação

que recebe um objeto de fila variável

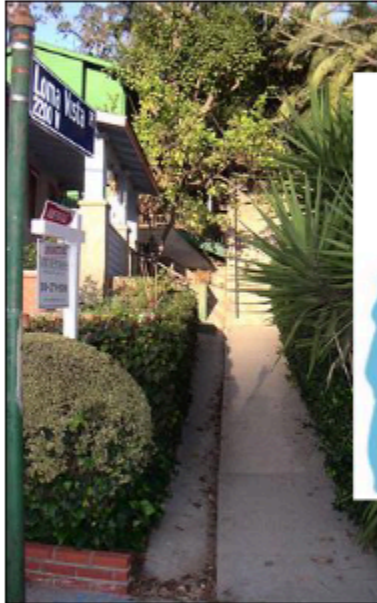


Busca e Modelos

- A busca opera sobre Modelos do Mundo
 - O agente não
- Na verdade, tente todos os planos no mundo real!
- Planejar é tudo "dentro simulação" ▪ Sua busca é apenas como
- Bons como seus modelos...



Busca que deu errado?



Busca que deu errado?

